

MSc. in Automotive Embedded Systems

Internship Final Oral Presentation

18-September-2025



**DARCY: Design and Development of an
Underwater Event Detection System Based on
Embedded AI**



- **Context & Objectives**
- **Enterprise Presentation**
- **State of the Art & Feasibility Study**
- **Missions**
- **Issue**
- **Results & Evaluation**
- **Lessons Learned**
- **Conclusion**



- **Context & Objectives**
- Enterprise Presentation
- State of the Art & Feasibility Study
- Missions
- Issue
- Results & Evaluation
- Lessons Learned
- Conclusion



Context

- The 6-month internship at ENIB, Lab-STICC, focused on designing an underwater event detection system using Embedded AI for autonomous marine drones.
- Enables real-time recognition of events (e.g., spotting fish, detecting trash, or visibility changes) without human intervention or internet connectivity.
- Challenges addressed: Limited communication, battery constraints, and high off-board compute costs.
- Contributed to marine research and sustainable ocean monitoring via optimized, edge-based AI deployment.



Objectives

- Prepare and label the Fish Recognition Ground Truth Dataset, arranging it for training and selecting an effective training strategy.
- Train and fine-tune the model using the Fish Recognition Ground Truth Dataset with techniques like Quantization.
- Deploy and optimize the trained YOLO models for underwater event detection on embedded hardware, targeting Raspberry Pi 5 with Pi Cam for real-time performance
- Evaluate and compare performance metrics (inference time, RAM usage, accuracy, model size, FPS) across all deployed YOLO variants on Raspberry Pi 5



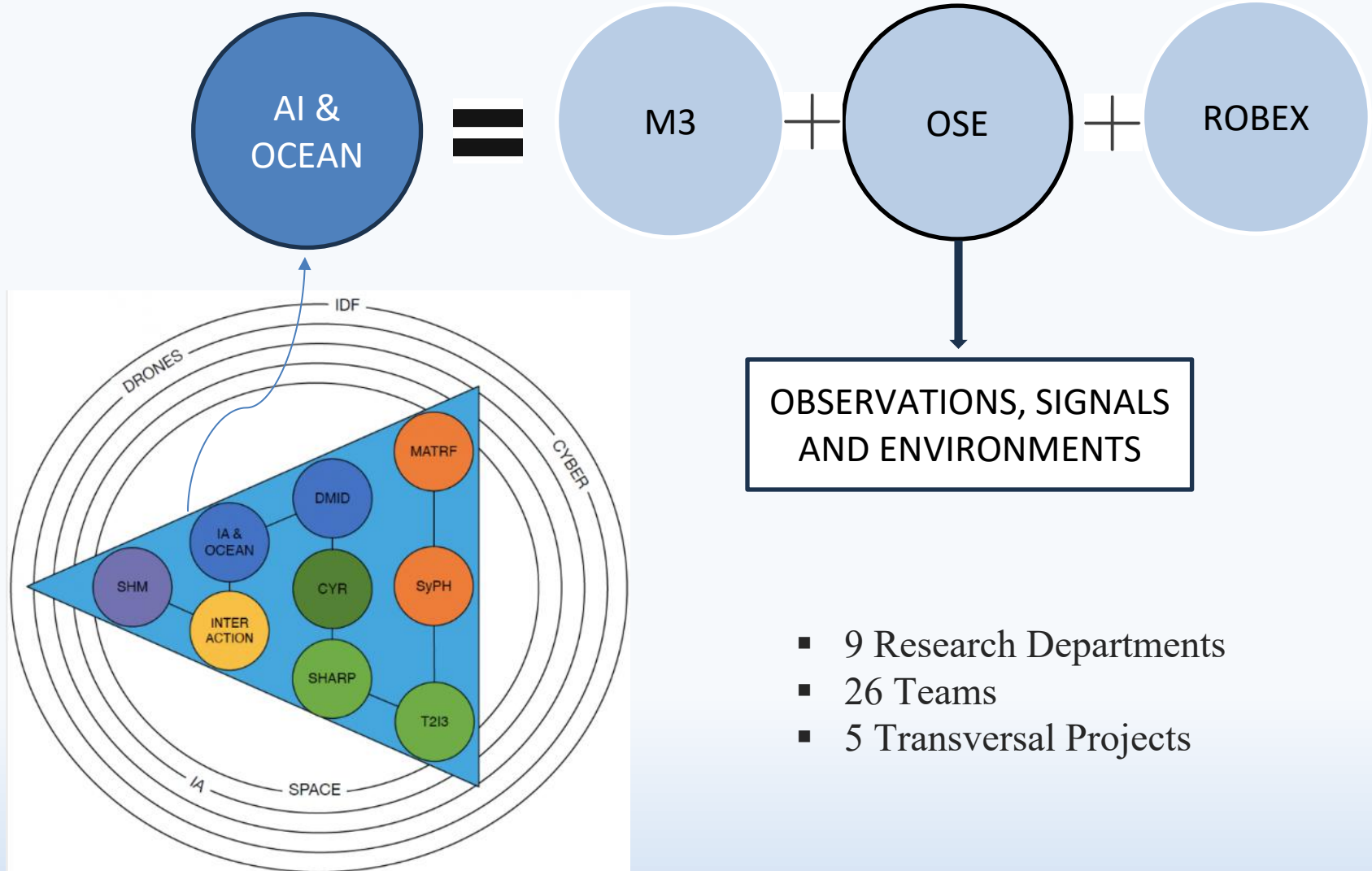
- Context & Objectives
- **Enterprise Presentation**
- State of the Art & Feasibility Study
- Missions
- Issue
- Results & Evaluation
- Lessons Learned
- Conclusion



Overview of Lab-STICC

- Laboratory for Science & Technology in Information, Communication & Knowledge.
- CNRS UMR 6285, created in 2008 (merger of ICT labs).
- ~650 researchers, engineers & PhD students
- Research areas: electronics, signal processing, AI, cybersecurity, robotics
- One of France's largest ICT research laboratories (CNRS-affiliated)
- Main Locations: Brest (ENIB, UBO, ENSTA), Lorient, Vannes, Quimper

Lab-STICC Divisions & Host Team





- Context & Objectives
- Enterprise Presentation
- **State of the Art & Feasibility Study**
- Missions
- Issue
- Results & Evaluation
- Lessons Learned
- Conclusion



Introduction

- **What is underwater event detection?**

→ Using AI to identify objects/events (fish, debris, luminosity changes) in underwater videos.

- **Why is this important?**

→ Supports marine science, pollution monitoring, biodiversity conservation, and autonomous underwater vehicles (AUVs/ROVs)

Challenges in Underwater AI

- **Dynamic environment** → changing light, moving debris, occlusions.

- **Limited datasets** → fewer annotated images compared to COCO/ImageNet.

- **Image quality** → low visibility, color distortion, turbidity.

- **Hardware limits** → embedded devices (Raspberry Pi) have low power vs. GPUs.



i)Blurry image



ii)occlusion

Comparison Table and Approach

Approach	Accuracy	Speed	Embedded Suitability	Remarks
Faster R-CNN	High	Low	×	Too heavy
SSD / EfficientDet	Medium	High	✓	Weak on small fish
YOLO (v5–v11)	High	High	✓✓	Best compromise

Gap & Positioning

- Most studies → GPU-based, not real-time.
- Missing link between research and **field deployment**.
- **This project:**
 - Uses **YOLOv11** for underwater fish/debris detection.
 - Optimized for **Raspberry Pi 5** (real-time, low energy, low cost).



+

YOLO

+

Under water Drone



For underwater event detection, typical metrics are:

Metric	What it shows	Why it matters
Precision	% of detected objects that are correct	Avoids false alarms (e.g., misclassifying debris as fish)
Recall	% of real objects detected	Ensures nothing important is missed
mAP (mean Average Precision)	Combines precision & recall across all classes	Standard in object detection benchmarks
FPS / Latency	Frames processed per second or detection time	Shows real-time capability, crucial for Raspberry Pi or drones



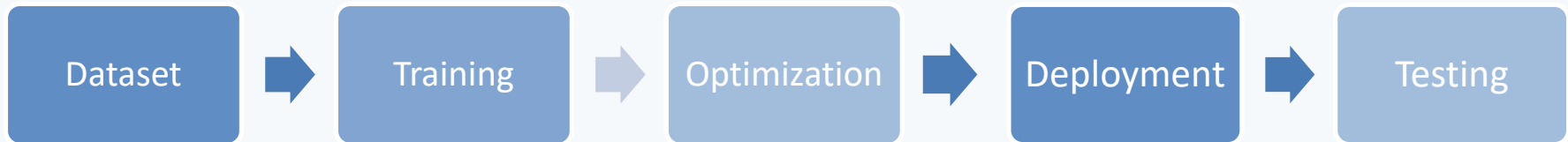
- Context & Objectives
- Enterprise Presentation
- State of the Art & Feasibility Study
- **Missions**
- Issue
- Results & Evaluation
- Lessons Learned
- Conclusion

- **Lab-STICC's mission** is to develop an underwater event detection system using embedded AI for real-time fish detection in marine environments.
- Challenges include underwater image distortions (light absorption, turbidity, occlusions), limited datasets, and resource constraints on embedded hardware.
- Focus on adapting Deep learning models for low-power platforms to enable autonomous monitoring with AUVs/ROVs.
- Collaboration between SHAKER (hardware/software optimization) and OSE (AI for environmental data) teams at ENIB, Brest.
- Project supports EUR ISblue initiative for interdisciplinary blue planet research, enhancing biodiversity monitoring and pollution detection.
- Deploy prototype on Embedded Platform for live inference, balancing accuracy and real-time performance.



Introduction and Scope

- Focus: Embedded AI for underwater event detection
- Application: Fish detection for marine monitoring
- Scope:



- Hardware: Raspberry Pi 5 + Pi Cam

•Tools & Platforms:

- Python, Google Colab, VS Code
- PyTorch, ONNX, TensorFlow Lite,
- OpenCV, ENIB local server

V-Model Development Process for Internship

- Understand underwater detection challenges
- Define objectives (real-time, embedded, robust)

System Requirements:

- Dataset preparation (fish images, masks, annotations)
- Define model architecture (YOLOv11 variants)

Functional Design

- Preprocessing & augmentation pipeline
- Training strategies (progressive augmentation, epochs,

Technical Design

Model Development

- Train YOLOv11 models
- Convert to TFLite / ONNX for deployment

Unit Testing

- Evaluate accuracy, precision, recall on test dataset
- Compare different YOLOv11 variants

Integration Testing:

- Deploy quantized models on Raspberry Pi 5
- Optimize inference pipeline (OpenCV + TFLite/ONNX)

System Validation

- Real-time fish detection with PiCam
- Performance evaluation

Tools and Development Environment

- Training: **ENIB local server** (GPU resources for deep learning),
- Development: **VS Code** for coding & debugging
- Deployment: **Raspberry Pi 5 + PiCam**
- Frameworks: **PyTorch** → **ONNX** → **TensorFlow Lite**
- Libraries: OpenCV, NumPy, Onnx runtime, TensorFlow Lite Runtime



Raspberry Pi 5 + Pi cam setup

Tools and Development Environment

```

Windows PowerShell  X  akshara@vision: ~  X  +  v

all      5298      6624      0.882      0.816      0.902      0.649

Epoch   GPU_mem   box_loss   cls_loss   dfl_loss   Instances   Size
51/150   8.71G     0.5615     0.2362     1.246      6           640: 100% 663/663 6.7it/s 1:40
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 5.2it/s 15.8s
all      5298      6624      0.882      0.815      0.901      0.648

Epoch   GPU_mem   box_loss   cls_loss   dfl_loss   Instances   Size
52/150   8.71G     0.5608     0.2401     1.244      8           640: 100% 663/663 6.9it/s 1:36
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 5.4it/s 15.3s
all      5298      6624      0.882      0.816      0.901      0.648

Epoch   GPU_mem   box_loss   cls_loss   dfl_loss   Instances   Size
53/150   8.71G     0.5609     0.2375     1.243      9           640: 100% 663/663 6.9it/s 1:36
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 5.4it/s 15.3s
all      5298      6624      0.882      0.816      0.902      0.648

Epoch   GPU_mem   box_loss   cls_loss   dfl_loss   Instances   Size
54/150   8.71G     0.5583     0.2359     1.243      8           640: 100% 663/663 6.9it/s 1:36
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 5.4it/s 15.3s
all      5298      6624      0.883      0.816      0.902      0.648

Epoch   GPU_mem   box_loss   cls_loss   dfl_loss   Instances   Size
55/150   8.71G     0.557      0.2358     1.243      5           640: 100% 663/663 6.8it/s 1:37
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 5.3it/s 15.6s
all      5298      6624      0.88      0.819      0.901      0.648

EarlyStopping: Training stopped early as no improvement observed in last 50 epochs. Best results observed at epoch 5, best model saved as best.pt
To update EarlyStopping(patience=50) pass a new patience value, i.e. `patience=300` or use `patience=0` to disable EarlyStopping.

55 epochs completed in 1.731 hours.
Optimizer stripped from fish_training/yolo11s_phase2_finetune/weights/last.pt, 19.2MB
Optimizer stripped from fish_training/yolo11s_phase2_finetune/weights/best.pt, 19.2MB

Validating fish_training/yolo11s_phase2_finetune/weights/best.pt...
Ultralytics 8.3.191 Python-3.8.10 torch-2.4.1+cu121 CUDA:2 (NVIDIA A40, 45449MiB)
YOLO11s summary (fused): 100 layers, 9,413,187 parameters, 0 gradients, 21.3 GFLOPs
Class    Images    Instances  Box(P      R           mAP50      mAP50-95): 100% 83/83 2.9it/s 28.7s
all      5298      6624      0.864      0.816      0.899      0.635

Speed: 0.1ms preprocess, 3.4ms inference, 0.0ms loss, 0.5ms postprocess per image
Results saved to fish_training/yolo11s_phase2_finetune
akshara@vision:~/newfish$

```

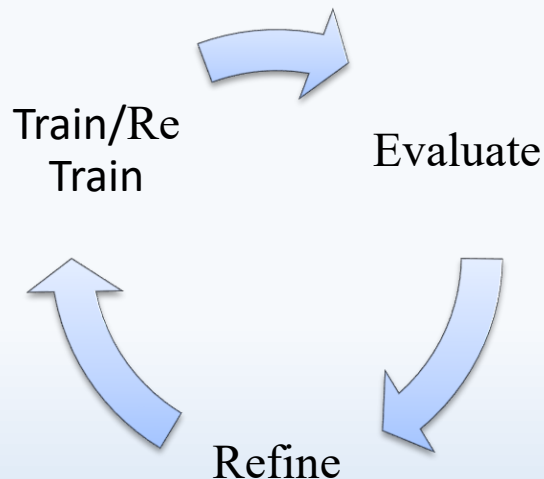
Example of training done in the ENIB Server

Planning and Workflow

•Stepwise workflow:



•Iterative approach:



• Parallel activities:

- Dataset curation & model training often overlapped
- Deployment tests started early in parallel with training

•Progress monitoring:

- Weekly meetings with supervisors(Every Week Wednesday)

Testing and Validation Strategy

•Unit Testing approach:

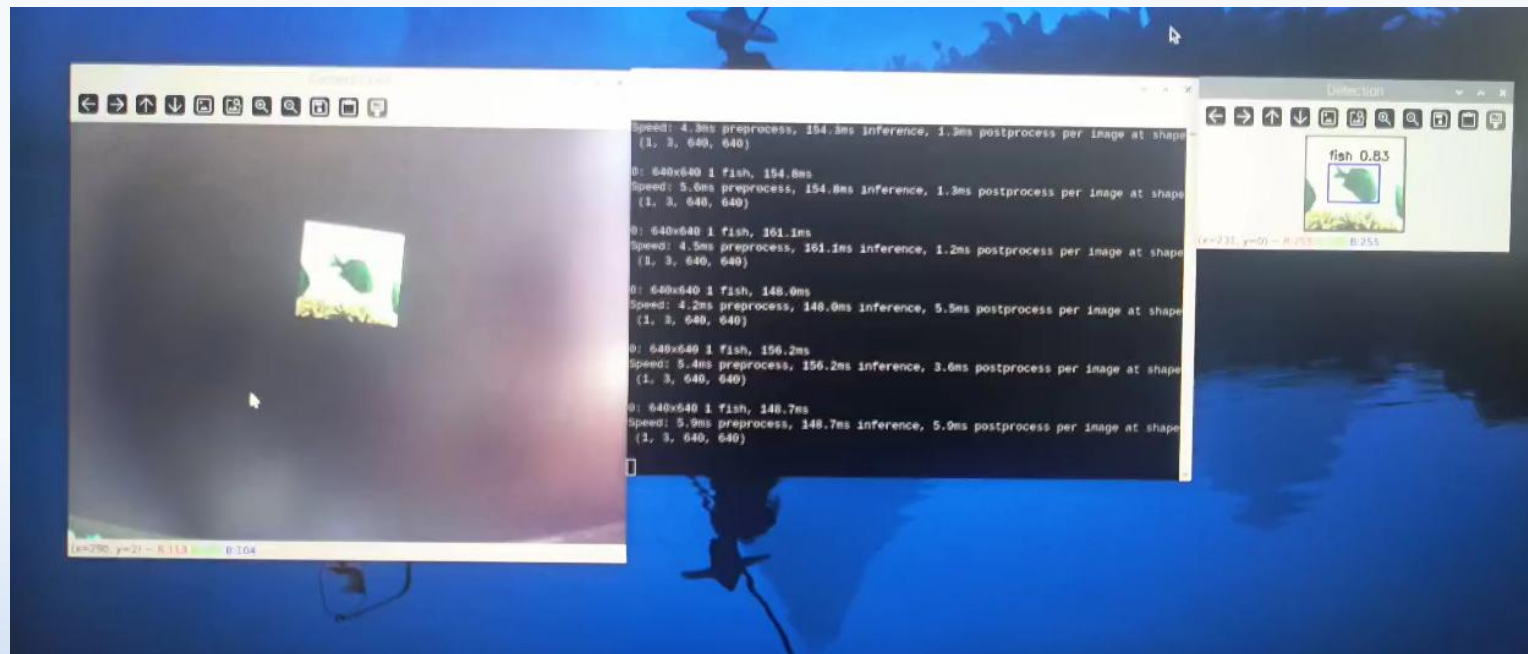
- Evaluated models on test datasets
- Metrics considered: Accuracy, Precision, Recall, mAP

•Integration Testing approach:

- Converted models (PyTorch → ONNX/TFLite)
- Deployed on Raspberry Pi 5
- Tested its performance

•System Validation approach:

- Ran live PiCam detection tests
- Measured inference speed (FPS), latency, memory, model size



Documentation and Scientific Communication

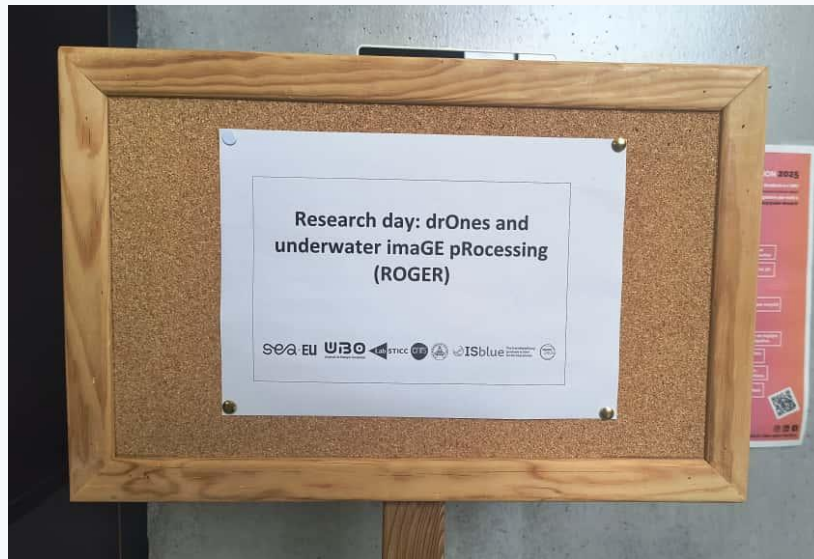
- Maintained structured documentation of datasets, training configs, and experiments
- Prepared detailed reports for internship supervisors



My workstation

Documentation and Scientific Communication

- Presented internship work at **ROGER(Research day:drOnes and underwater imaGe pRocessing)** research program, Brest (2025)





- Context & Objectives
- Enterprise Presentation
- State of the Art & Feasibility Study
- Missions
- **Issue**
- Results & Evaluation
- Lessons Learned
- Conclusion

Dataset Issues	Training Issues
Redundancy: consecutive video frames → low diversity	GPU memory limits with larger YOLOv11 models
Blurry frames from motion & turbidity	Overfitting due to limited dataset diversity
Occlusions by algae/rocks → hard to annotate	Unstable convergence with strong augmentation
Manual labeling effort → time-consuming, error-prone	Trade-off between resolution (accuracy) vs speed (640 vs 800)



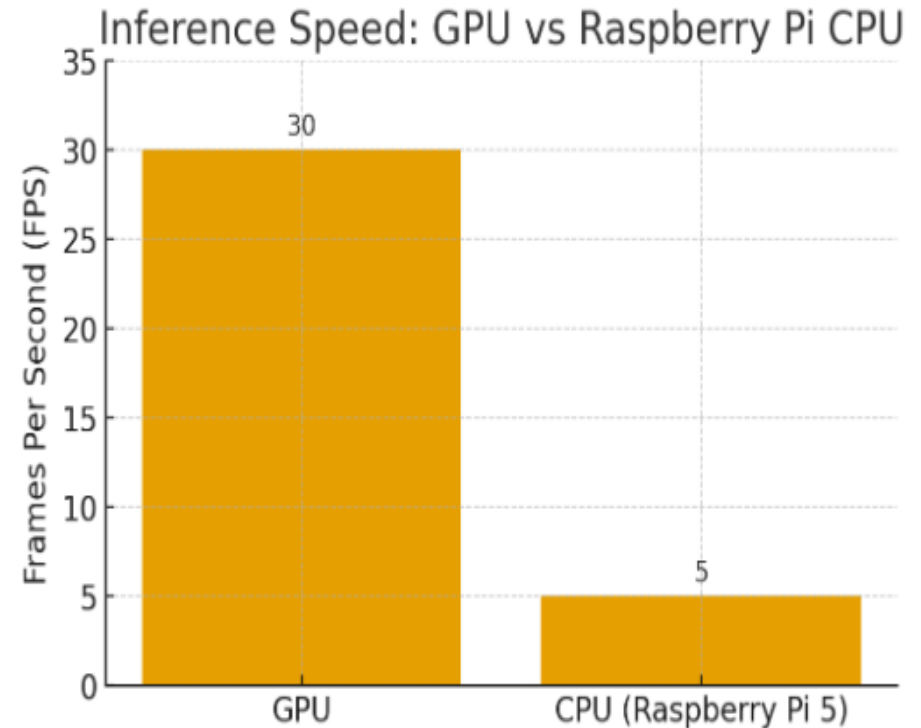
Blurred fish image



Clear fish image



Model Conversion & Optimization Issues	Hardware Issues (Raspberry Pi 5)
ONNX ↔ TFLite conversion conflicts (dependencies, versions)	Dependency installation conflicts (PyTorch, ONNX, OpenCV)
Quantization (FP16/INT8) sometimes reduced accuracy	CPU-only inference → low FPS compared to GPU
ARM64 compatibility issues on Raspberry Pi → manual fixes	SD card corruption due to repeated flashing & I/O load





- Context & Objectives
- Enterprise Presentation
- State of the Art & Feasibility Study
- Missions
- Issue
- **Results & Evaluation**
- Lessons Learned
- Conclusion

Training Results of different Yolo variants

Model Variant	Best Epoch	Precision	Recall	mAP50	mAP50-90
Yolo 11n	101	0.89	0.82	0.91	0.65
Yolo 11s(no phase Aug)	40	0.891	0.821	0.920	0.676
Yolo hybrid(n+s)	72	0.89	0.83	0.917	0.675
Yolo11s (with Aug)	76	0.901	0.813	0.934	0.68

Deployment and Testing of best.pt (Pytorch Model)

Model Variant	Size	Inference (ms)	FPS	mAP50	mAP50-90
Yolo 11n	5.21	387.3	2.76	0.98	0.864
Yolo 11s(no phase Aug)	18.28	1072.3	1.01	0.979	0.83
Yolo hybrid(n+s)	18.17	1136.4	0.87	0.983	0.88
Yolo11s (with Aug)	18.31	1675.7	0.66	0.984	0.903

Quantized TFLite Deployment- Resource Usage

Model Variant	Format	Size(MB)	FPS	Inference Time	Notes
Yolo 11n	int8	2.63	3.26	306.8	Best Tradeoff
Yolo 11s(no phase Aug)	int8	9.17	1.19	837.1	Balanced Slow
Yolo hybrid(n+s)	int8	9.0	1.19	841.5	Strong But Slow
Yolo11s (with Aug)	int8	9.18	1.20	832.4	Highest Accuracy

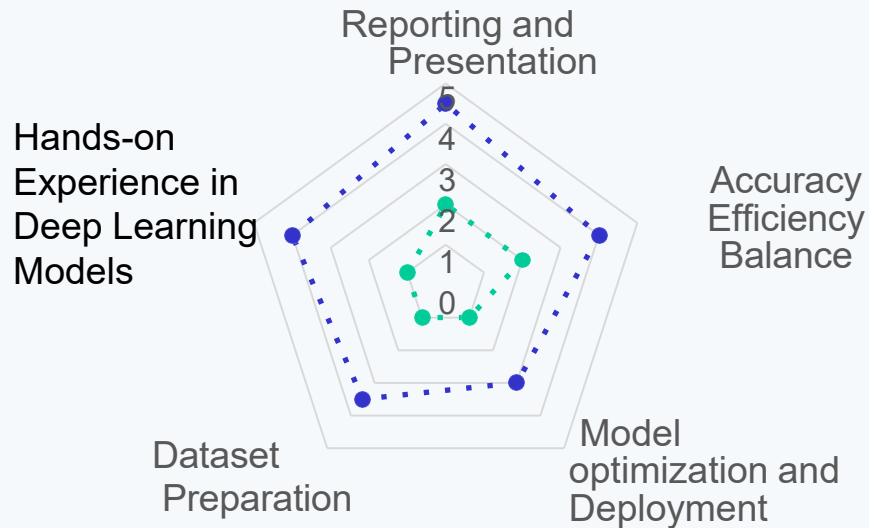


- Context & Objectives
- Enterprise Presentation
- State of the Art & Feasibility Study
- Missions
- Issue
- Results & Evaluation
- **Lessons Learned**
- Conclusion



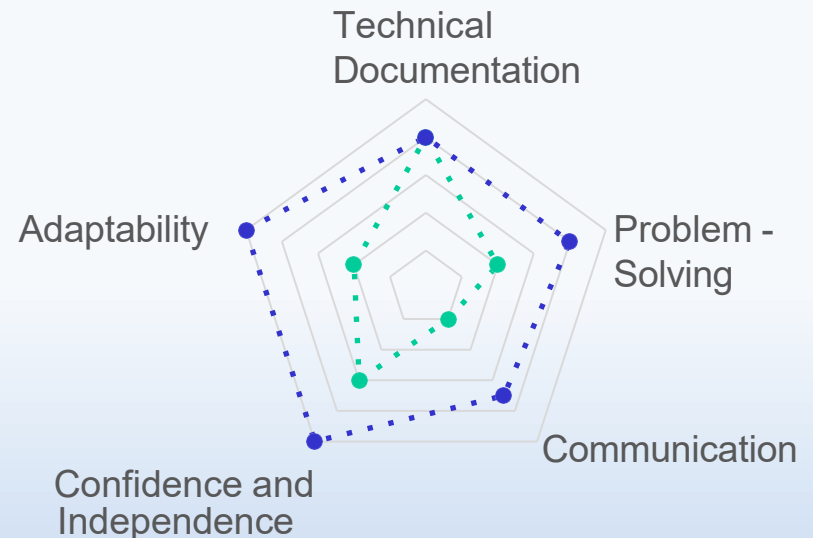
Professional Skills

Before Internship After Internship



Personal Skills

Before Internship After Internship





- Context & Objectives
- Enterprise Presentation
- State of the Art & Feasibility Study
- Missions
- Issue
- Results & Evaluation
- Lessons Learned
- **Conclusion**

- Contributed to the **development of an underwater event detection prototype**, focusing on fish detection using YOLOv11.
- Gained **hands-on experience** with dataset preparation, training strategies, quantization, and deployment on Raspberry Pi 5.
- Learned to handle **real-world challenges** in embedded AI, including hardware limits, optimization trade-offs, and testing constraints.
- Developed skills in **AI model design, system integration, and research reporting**.
- Grateful to the **Lab-STICC team and mentors** for their guidance and support, and motivated to apply these learnings to future projects in **embedded AI and environmental monitoring**.



THANK YOU!